Deep Generative Models

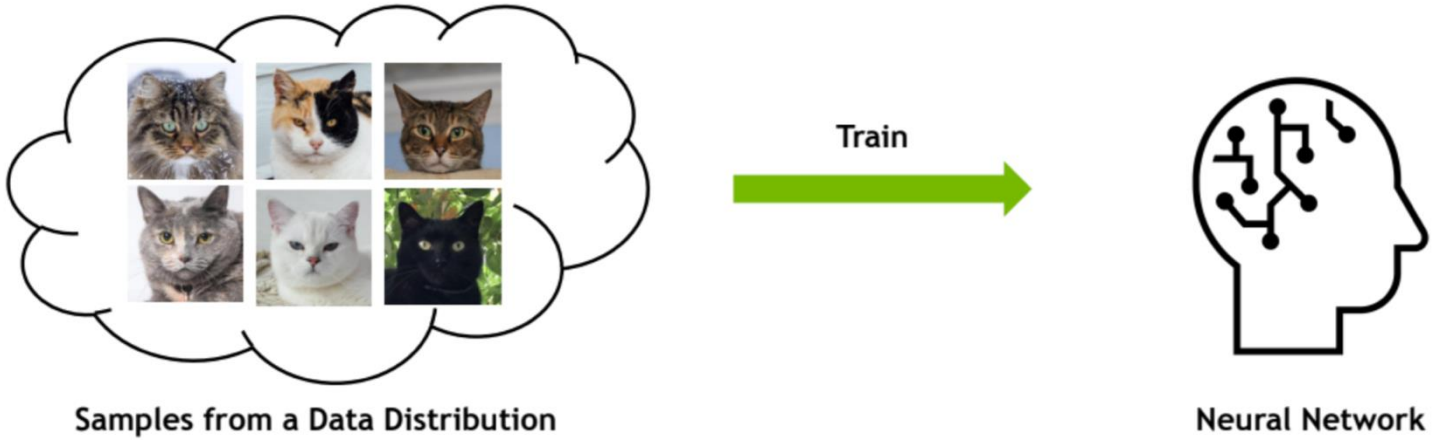
Jian Tang
HEC Montreal
Mila-Quebec AI Institute
CIFAR AI Research Chair
Email: jian.tang@hec.ca

Outline

- Introduction to Generative Models
- Variational Auto-Encoders (VAEs)
- Generative Adversarial Networks (GANs)
- Denoising Diffusion Models (DDMs)

Deep Generative Models

- Modeling the joint distribution $p(X)$, and drawing samples from it



Slide from https://github.com/cvpr2023-tutorial-diffusion-models

Application: (1) Image Generation

Slide from https://github.com/cvpr2023-tutorial-diffusion-models
Image from StyleGan3

Application: (2) Representation Learning

- Learning from limited labels

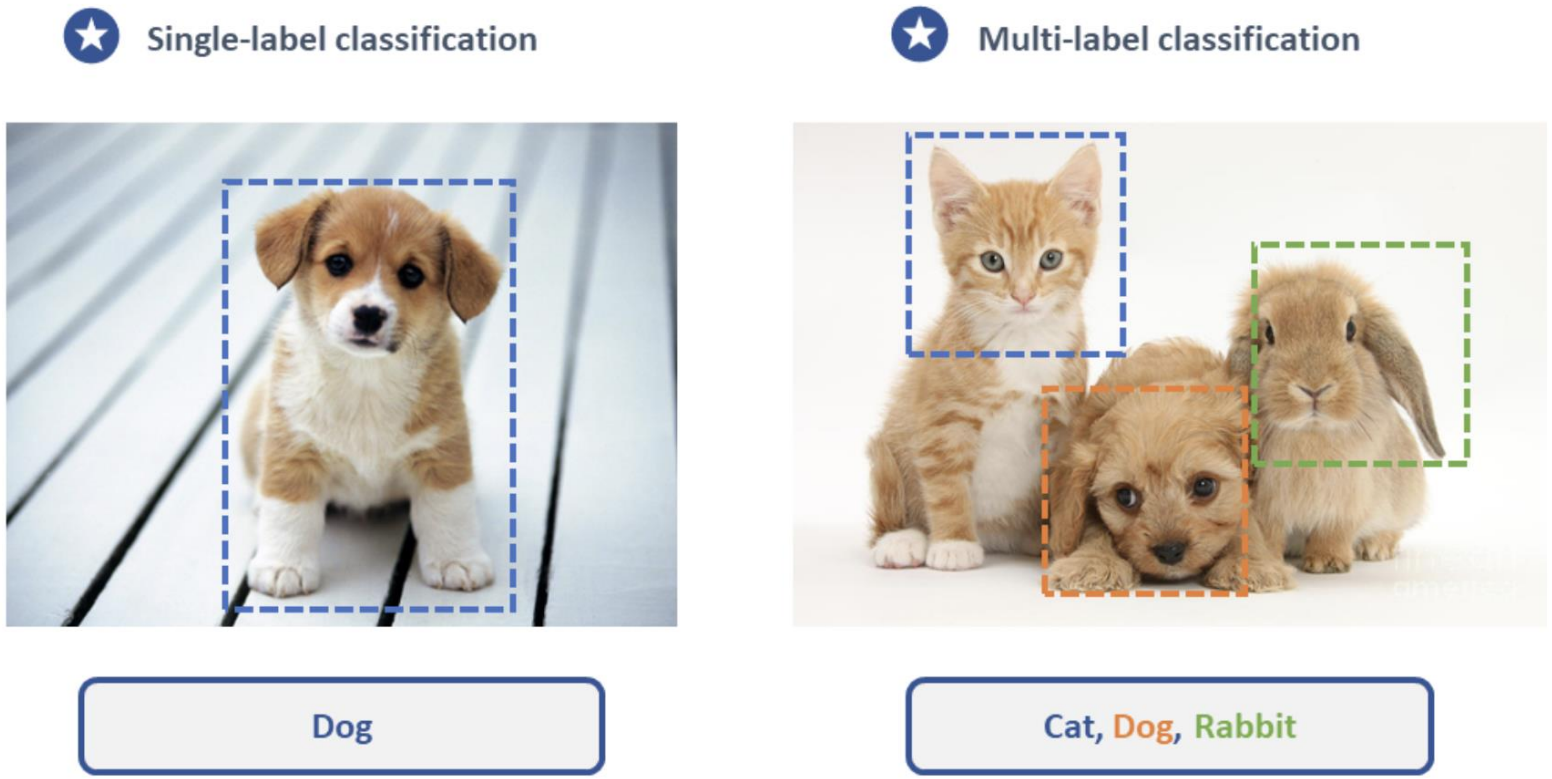


Image from https://research.aimultiple.com/image-classification/

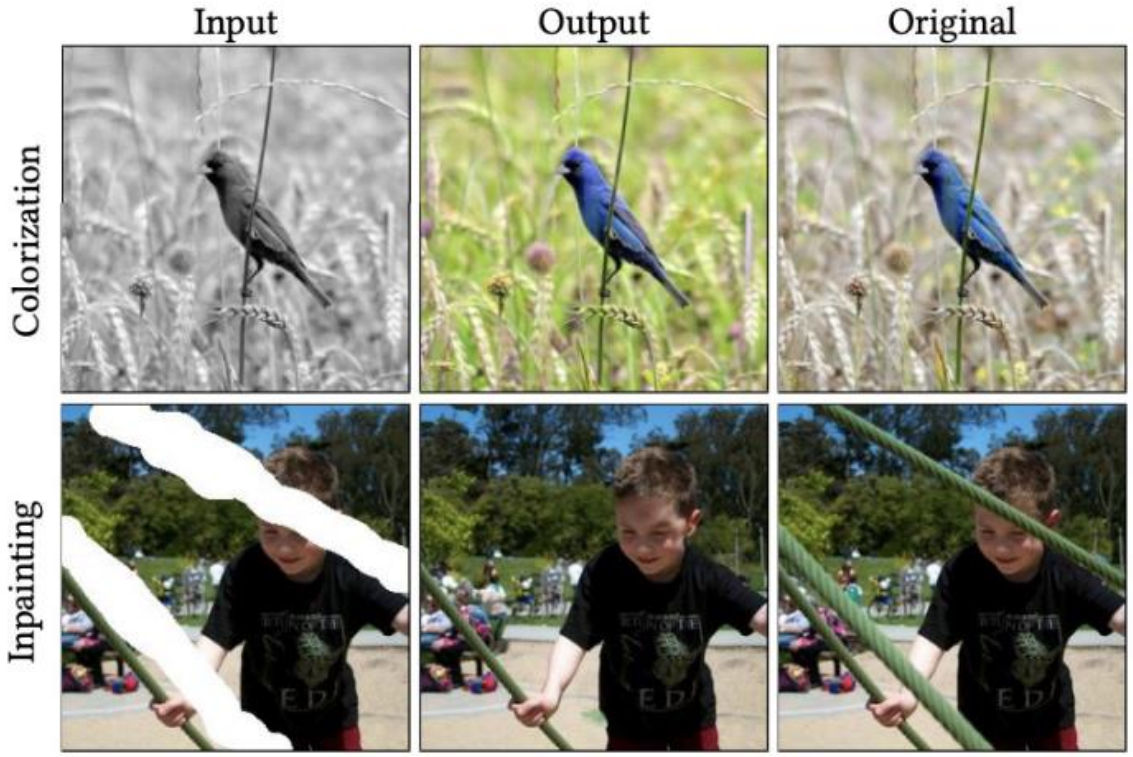
Application: (3) Super-resolution

Low Resolution

High Resolution

Image from https://heartbeat.comet.ml/super-resolution-580424fb7f7c

Application: (4) Colorization, Inpainting, Restoration



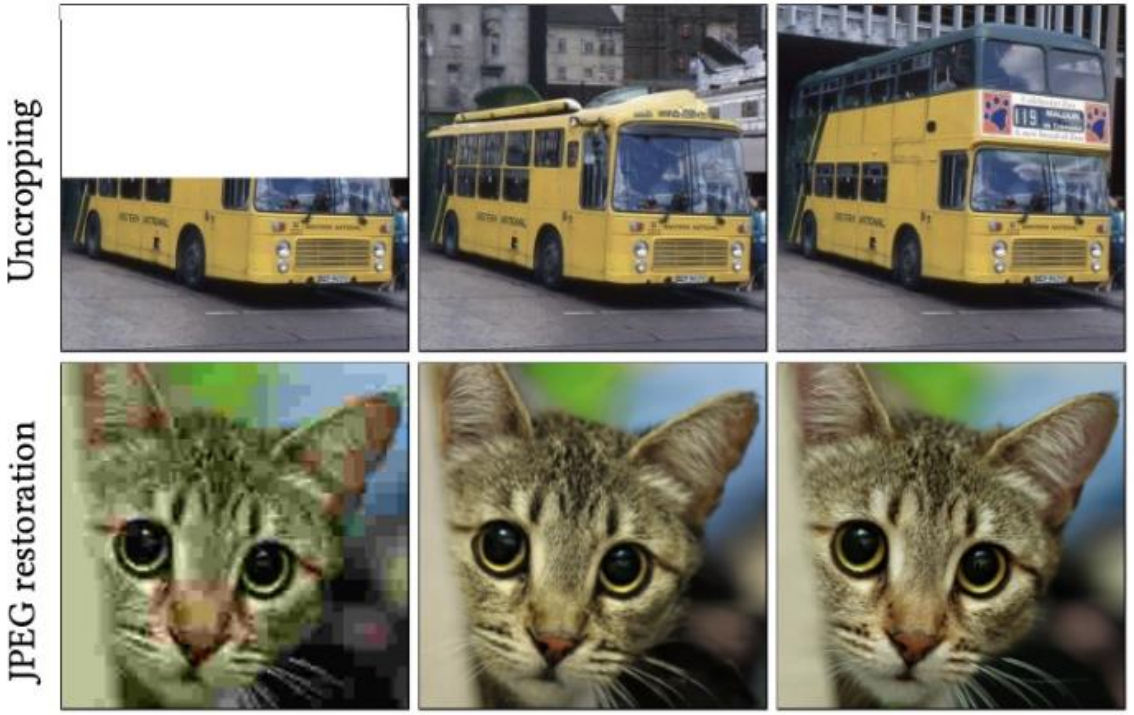


Image from Palette: Image-to-Image Diffusion Models https://arxiv.org/pdf/2111.05826

Application: (5) Outfilling

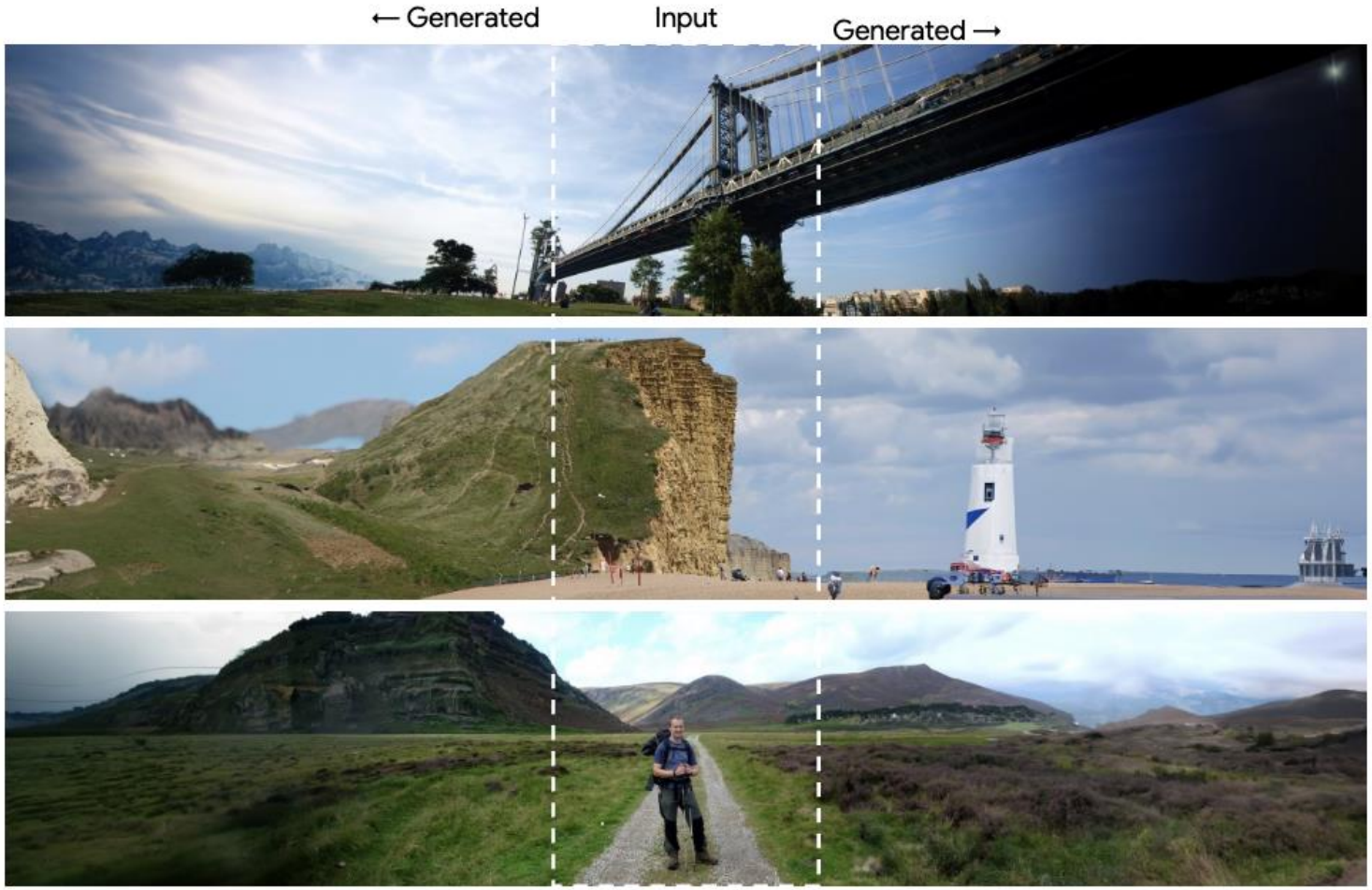


Image from Palette: Image-to-Image Diffusion Models https://arxiv.org/pdf/2111.05826

What Makes a Good Generative Model?

- The generative learning trilemma

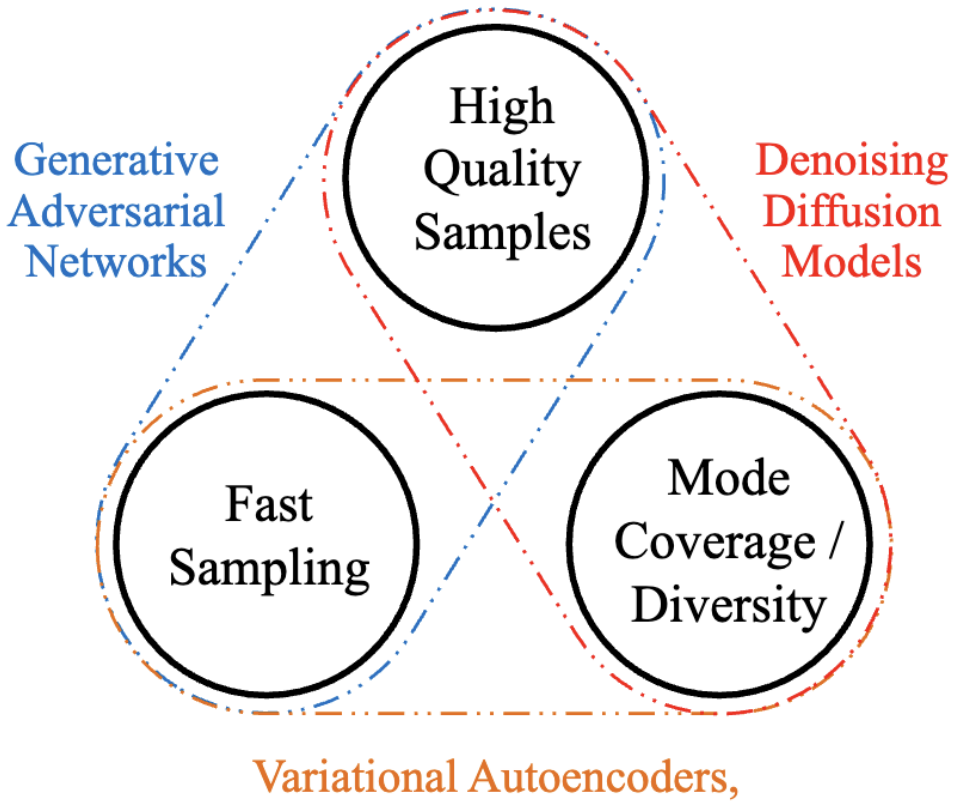
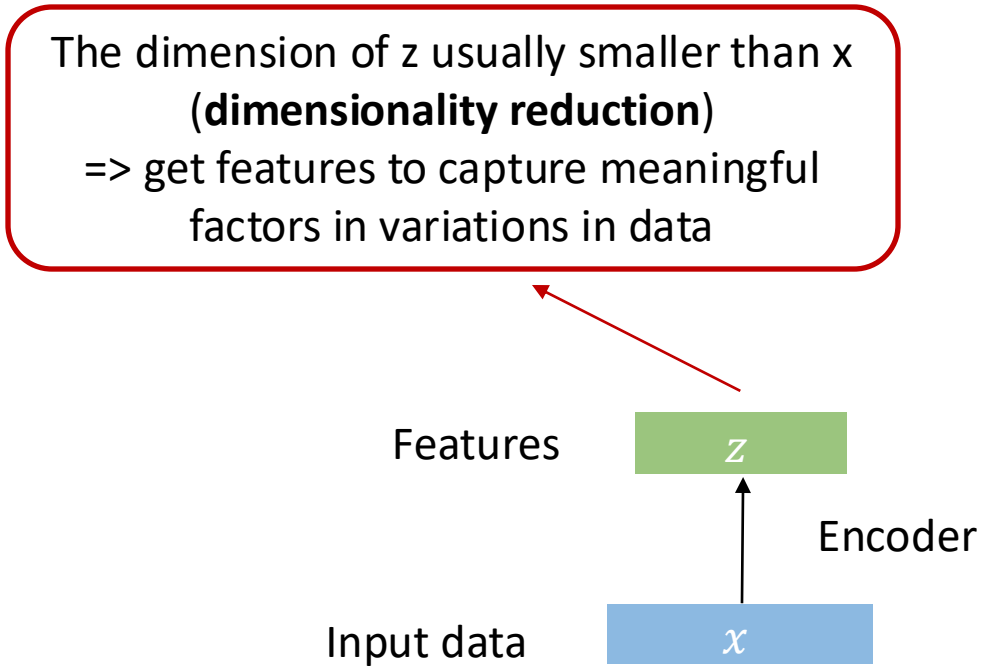


Image from Tackling the Generative Learning Trilemma with Denoising Diffusion GANs https://arxiv.org/pdf/2112.07804

Variational Auto-Encoders (VAEs)

Background: Auto-Encoders

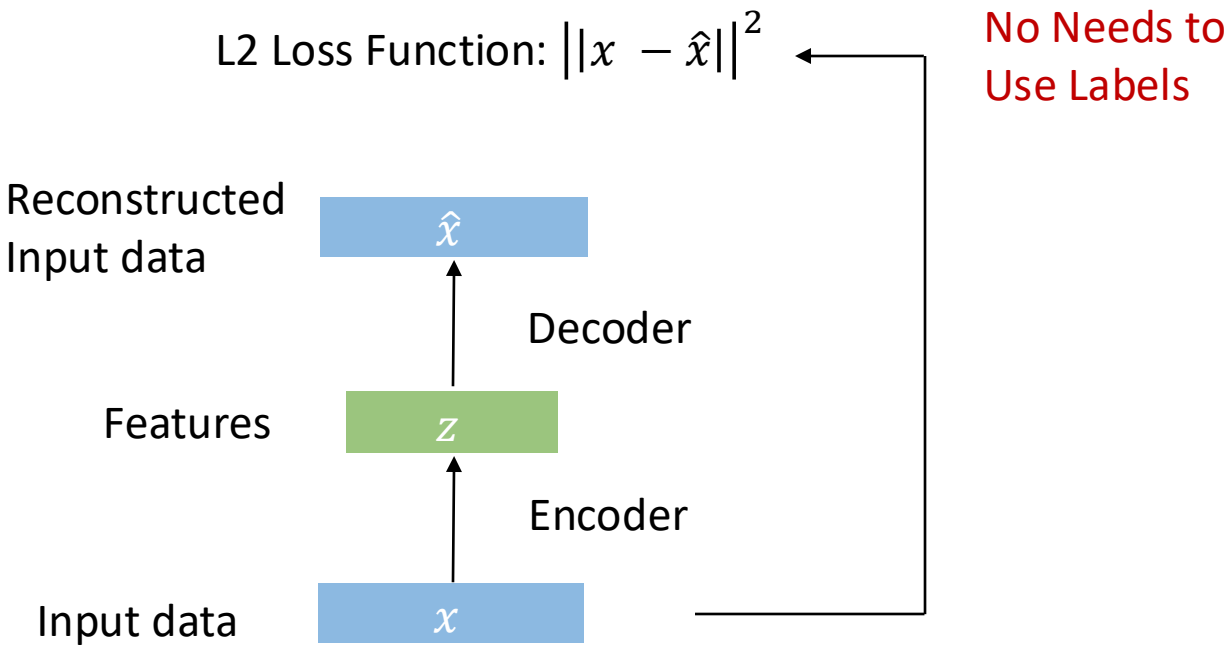
- Unsupervised approach for learning a low-dimensional feature representation from unlabeled training data



Slide adapted from https://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture13.pdf

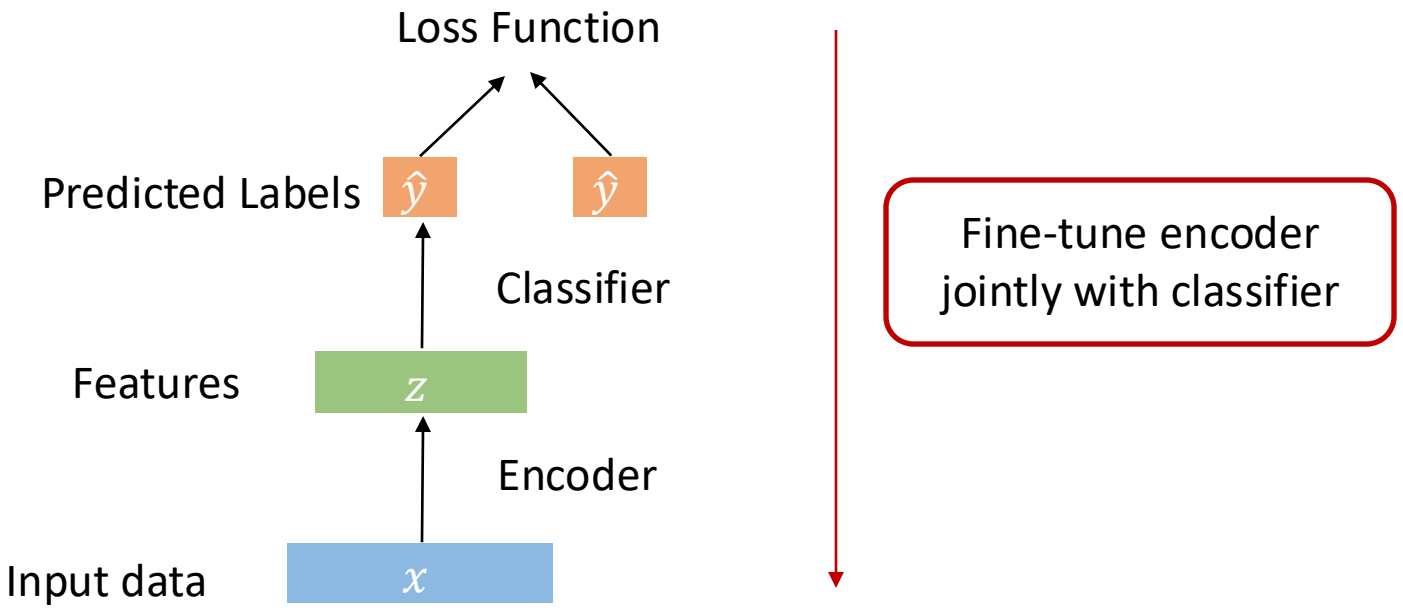
Background: Auto-Encoders

- How to learn this feature representation?
  - Train such that features can be used to reconstruct original data



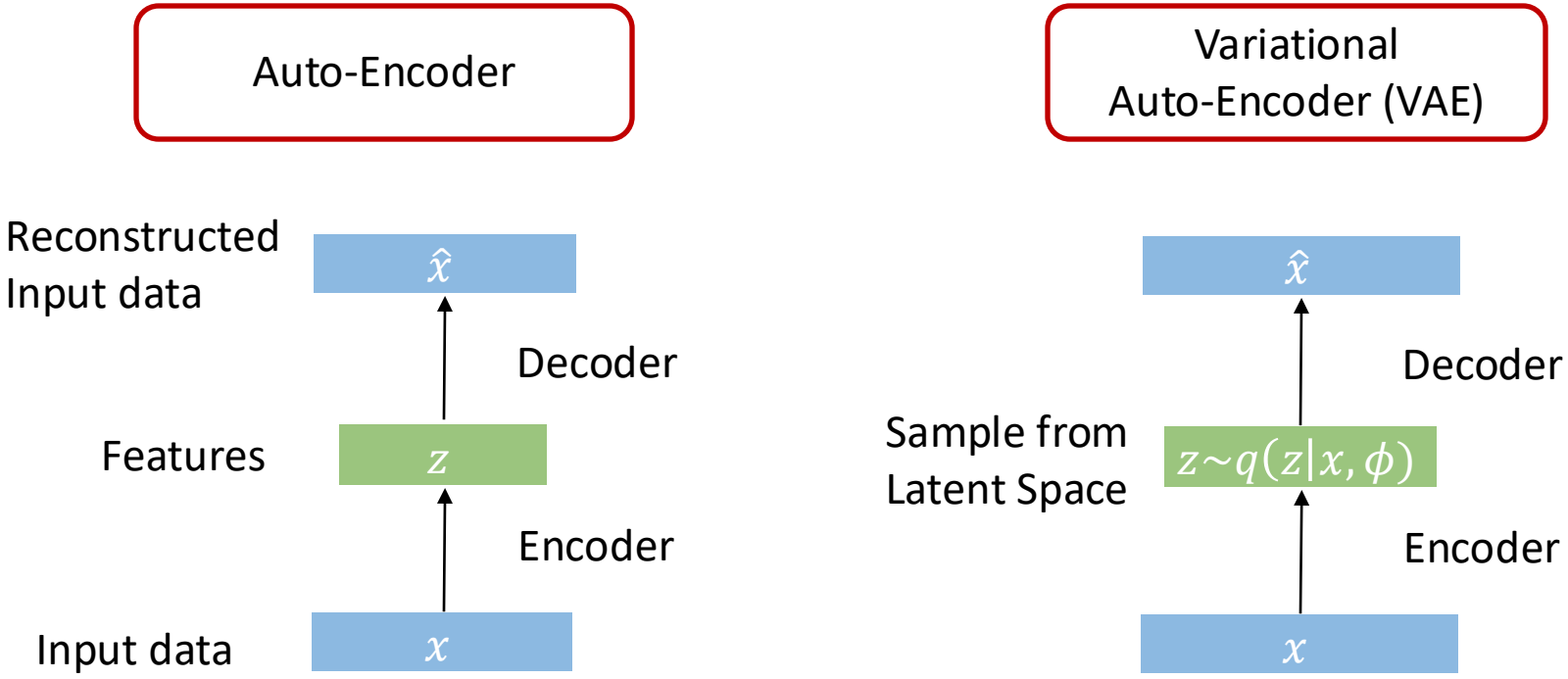
Background: Use Case of Auto-Encoders

- Encoder can be used to initialize a supervised model



Variational Auto-Encoder (VAE)

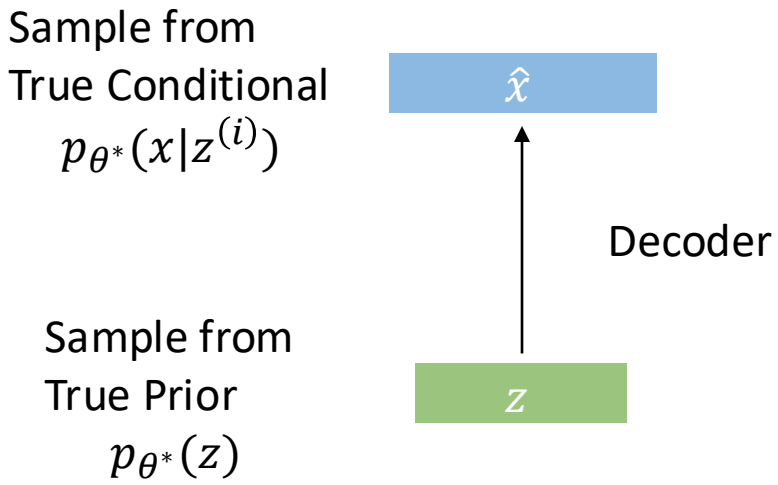
- **Can we generate new images from an auto-encoder?**
  - **=> Variational auto-encoder (VAE)**



Variational Auto-Encoder (VAE)

- Assume training data $\left\{x^{(i)}\right\}_{i=1}^{N}$ generated from underlying unobserved (latent) representation z

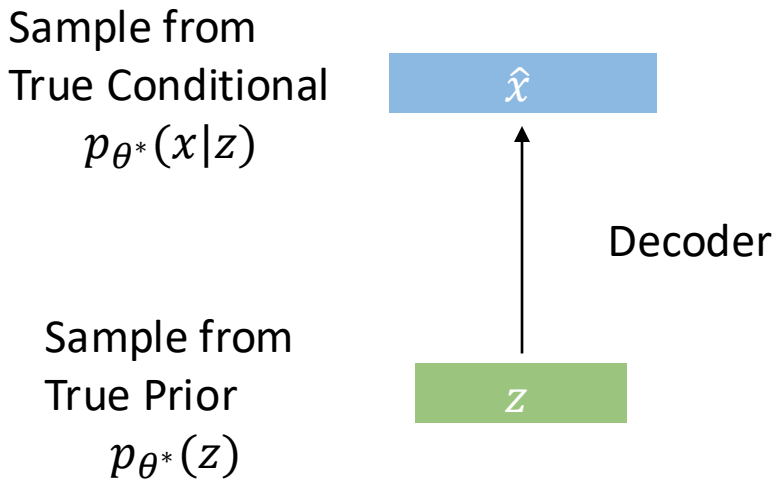
$$p(x) = p(z)p(x|z)$$



Variational Auto-Encoder (VAE)

- How to train the model?
  - Learn model parameters θ to maximize likelihood of training data
  - $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$

Intractable to compute $p_\theta(x|z)$ for every z



Variational Inference*

Jensen Inequality

$$\log \int p(x)g(x)dx \geq \int p(x) \log g(x)dx$$

$$\log p(x) \geq \int q(z|x) \log \left(p(x|z) \frac{p(z)}{q(z|x)} \right) dz$$

Tractable!

Evidence Lower Bound (ELBO)

$$= \int q(z|x) \log p(x|z)\, dz - \int q(z|x) \log \frac{q(z|x)}{p(z)}\, dz$$

Simple Gaussian Prior

$$\mathbb{E}_{q(z|x)}[\log p(x|z)] - KL[q(z|x)||p(z)]$$

*θ is ignored for simplicity

Parameterization of Posterior Distribution

- Choose the proposed distribution $q(z|x)$ as a neural network $q_\phi(z|x)$, modeling latents based on the observed input data

$$\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)}\boldsymbol{I})$$
$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I}) \quad ; \text{Reparameterization trick.}$$

where $\odot$ refers to element-wise product.

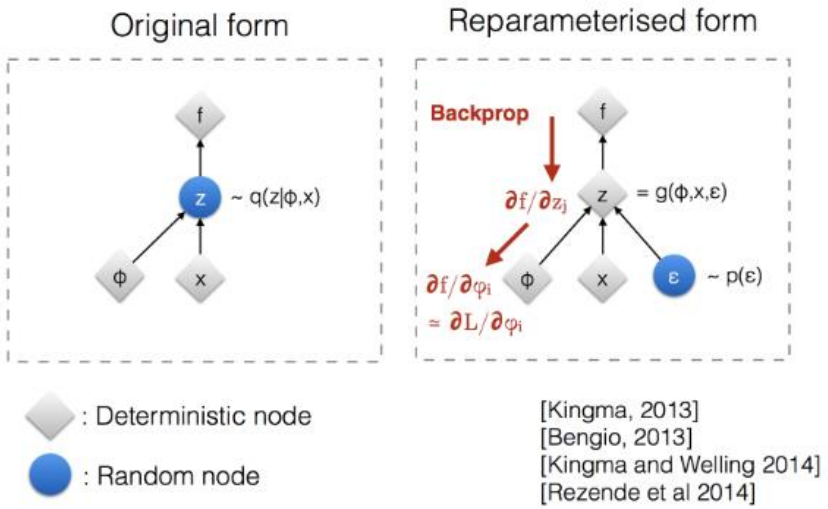
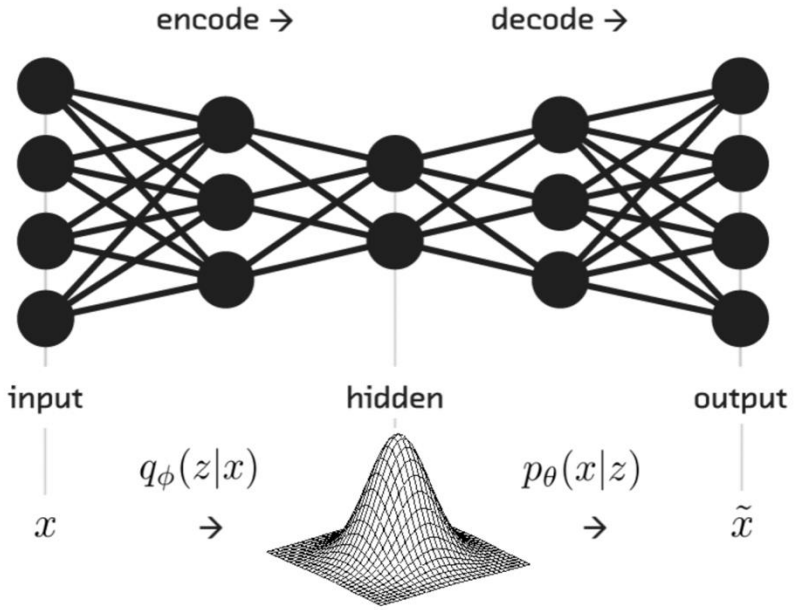


Figure from https://lilianweng.github.io/posts/2018-08-12-vae/

The Final ELBO Objective

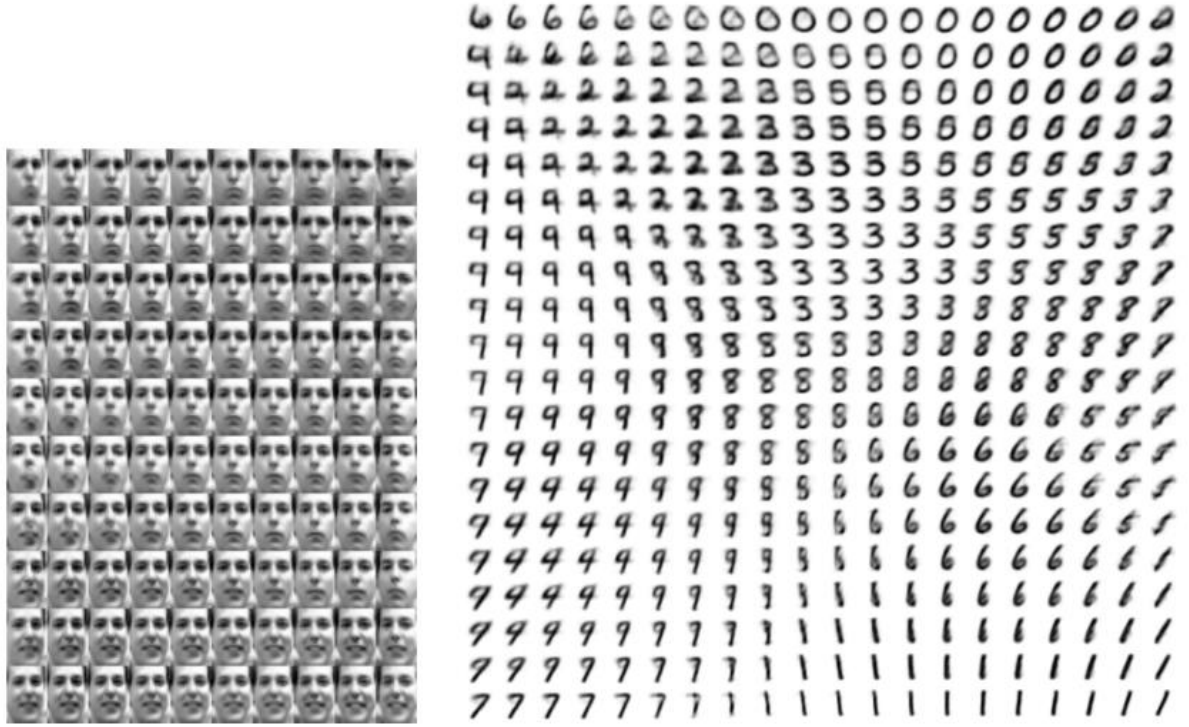


ELBO

$$\mathbb{E}_{\mathrm{q}_{\boldsymbol{\phi}}(\mathrm{z}|x)}[\log p_\theta(x|z)] - KL[q_\phi(z|x)||p(z)]$$

Reconstruction error **Distance between posterior and prior distributions**

VAEs for Image Generation



(a) Learned Frey Face manifold (b) Learned MNIST manifold

Figure 4: Visualisations of learned data manifold for generative models with two-dimensional latent space, learned with AEVB. Since the prior of the latent space is Gaussian, linearly spaced coordinates on the unit square were transformed through the inverse CDF of the Gaussian to produce values of the latent variables $\mathbf{z}$. For each of these values $\mathbf{z}$, we plotted the corresponding generative $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$ with the learned parameters $\boldsymbol{\theta}$.

Image from VAE: Auto-Encoding Variational Bayes https://arxiv.org/pdf/1312.6114

Open-source Code

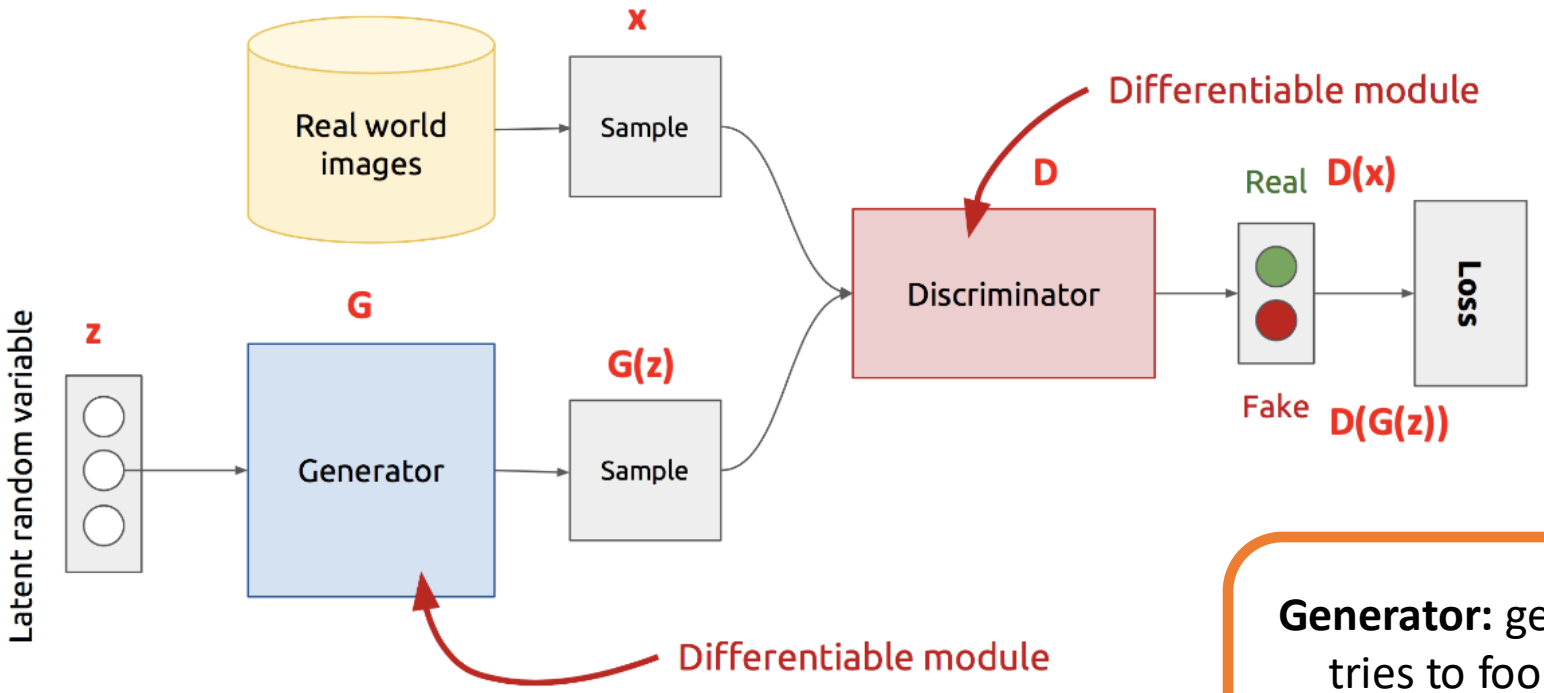
- https://colab.research.google.com/github/smartgeometry-ucl/dl4g/blob/master/variational_autoencoder.ipynb

Generative Adversarial Network (GAN)

Two Components

- **Generator**
  - A generator used for data generation
- **Discriminator**
  - Used to distinguish between real images and fake images generated from the generator

GAN Architecture

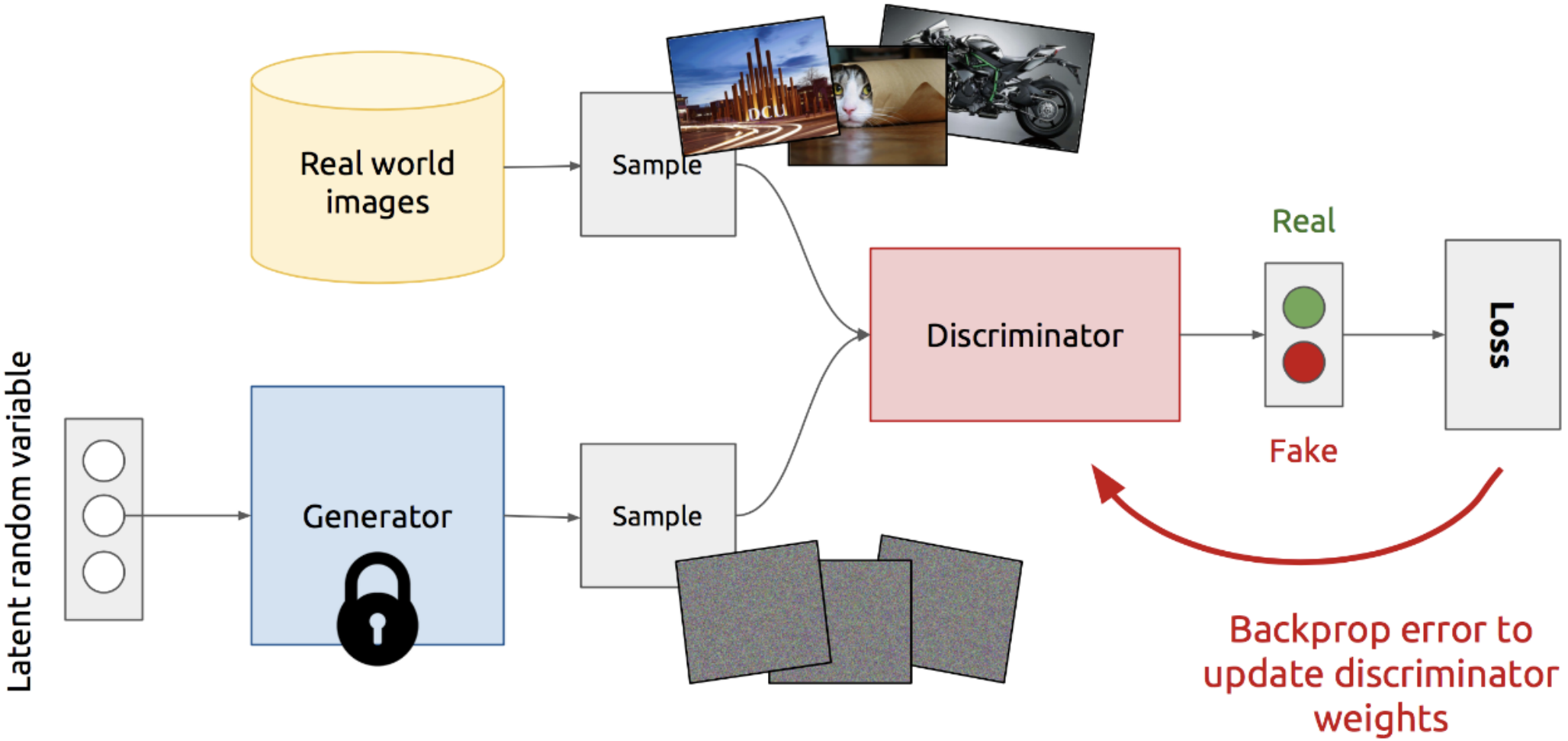


Generator: generate fake samples, tries to fool the Discriminator
Discriminator: tries to distinguish between real and fake samples

- **Z** is some random noise (Gaussian/Uniform).
- **Z** can be thought as the latent representation of the image.

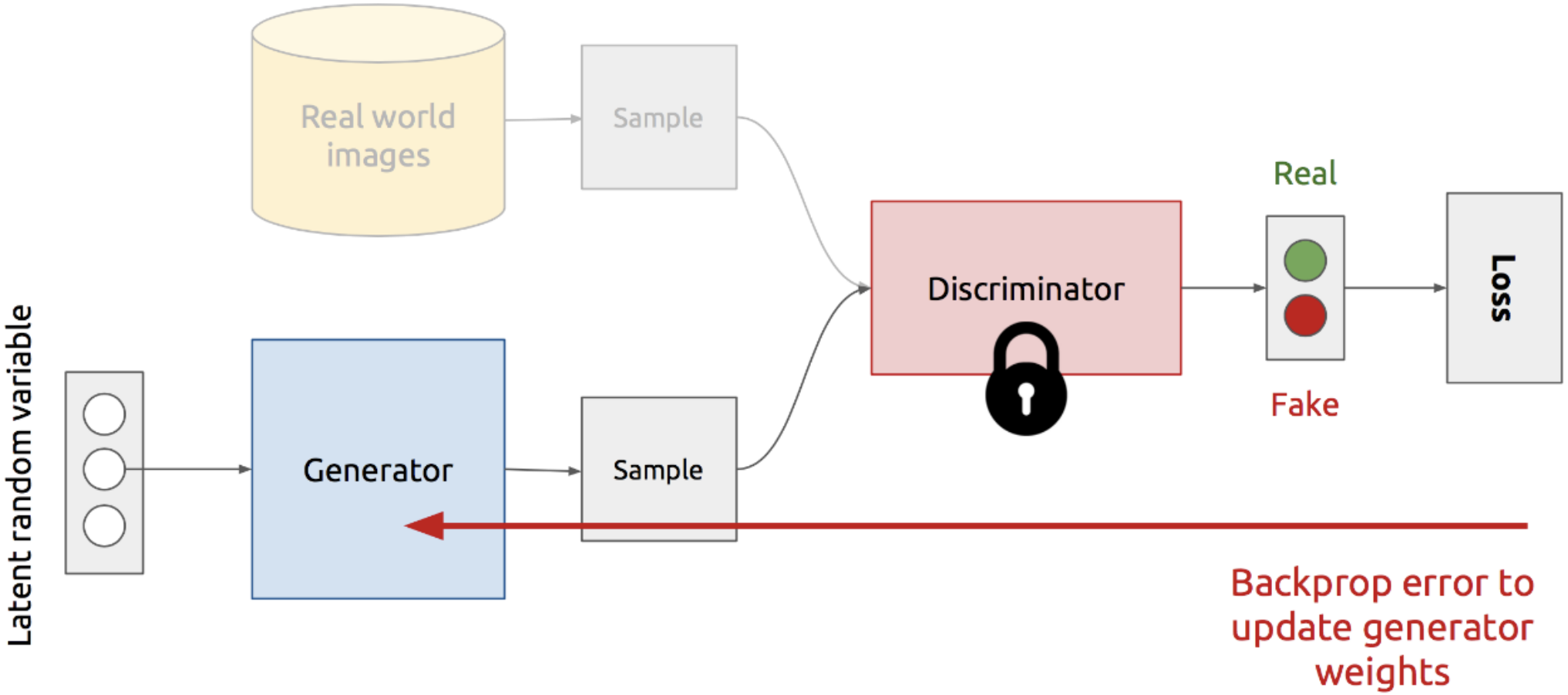
Slide from https://slazebni.cs.illinois.edu/spring17/lec11_gan.pdf

Training Discriminator



Slide from https://slazebni.cs.illinois.edu/spring17/lec11_gan.pdf

Training Generator



Slide from https://slazebni.cs.illinois.edu/spring17/lec11_gan.pdf

GAN's Formulation

$$\min_G \max_D V(D,G)$$

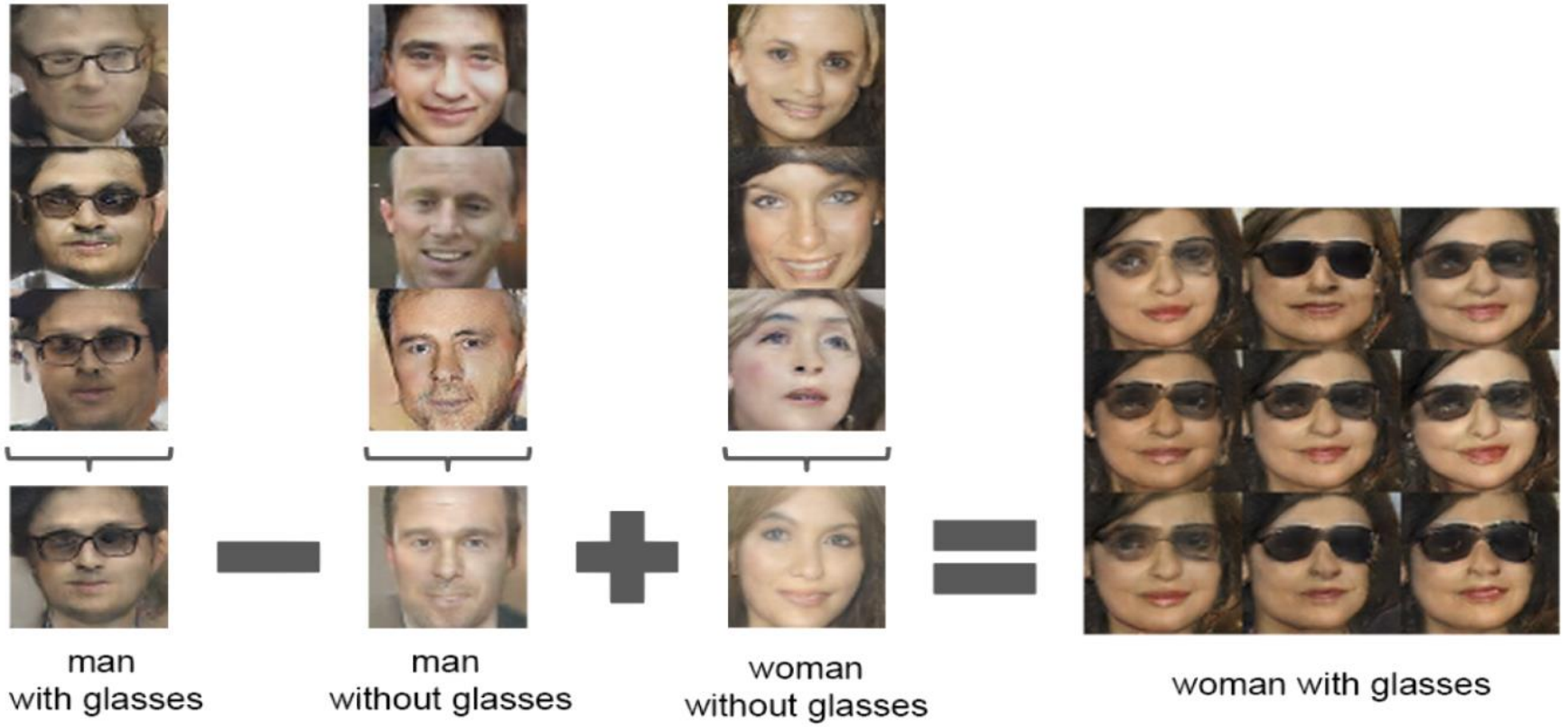
- It is formulated as a **minimax game**, where:
  - The Discriminator is trying to maximize its reward $\boldsymbol{V(D,G)}$
  - The Generator is trying to minimize Discriminator's reward (or maximize its loss)

$$V(D,G) = \mathbb{E}_{x\sim p(x)}[\log D(x)] + \mathbb{E}_{z\sim q(z)}\left[\log\left(1 - D(G(z))\right)\right]$$

- The Nash equilibrium of this particular game is achieved at:
  - $P_{data}(x) = P_{gen}(x) \;\; \forall x$
  - $\mathrm{D}(x) = \frac{1}{2} \;\; \forall x$

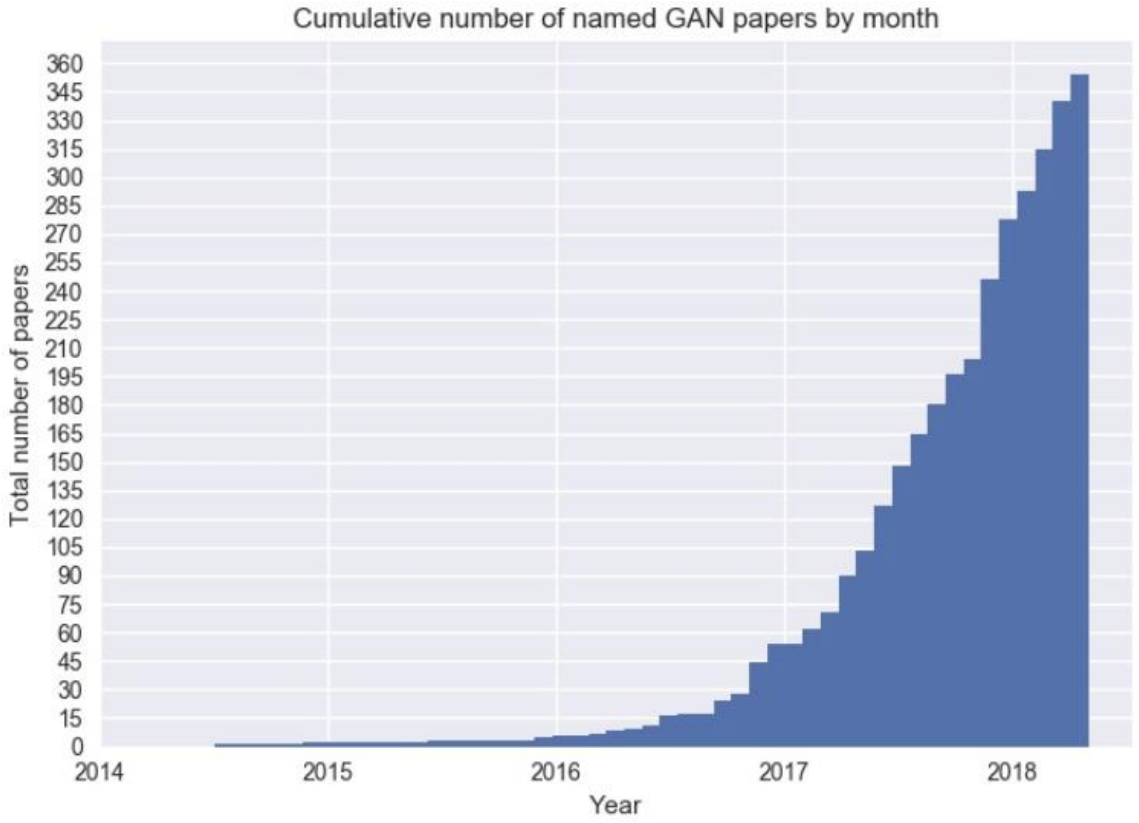
Slide from https://slazebni.cs.illinois.edu/spring17/lec11_gan.pdf

Latent Vectors Capture Interesting Patterns



Slide from Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks https://arxiv.org/abs/1511.06434

Track Updates at the GAN Zoo



https://github.com/hindupuravinash/the-gan-zoo

Slide from https://efrosgans.eecs.berkeley.edu/CVPR18_slides/Introduction_by_Goodfellow.pdf

Denoising Diffusion Model

The State-of-the-Art Generative Model

Image from Midjourney v4.

Why Diffusion?

- **VAE & GAN:**
  - Convert latent noise vector to target distribution in one step
- **Diffusion:**
  - Estimating and analyzing in small step sizes are more tractable and easier

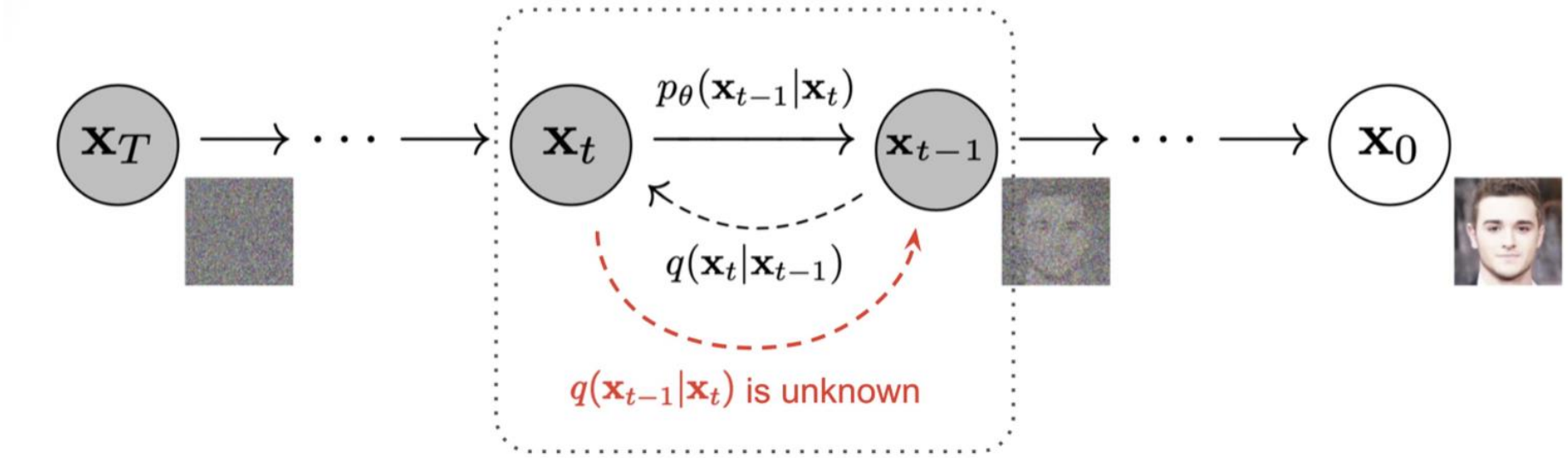
Forward diffusion process gradually adds noise to input

Reverse denoising process learns to generate data by denoising

Image from https://cvpr2022-tutorial-diffusion-models.github.io/

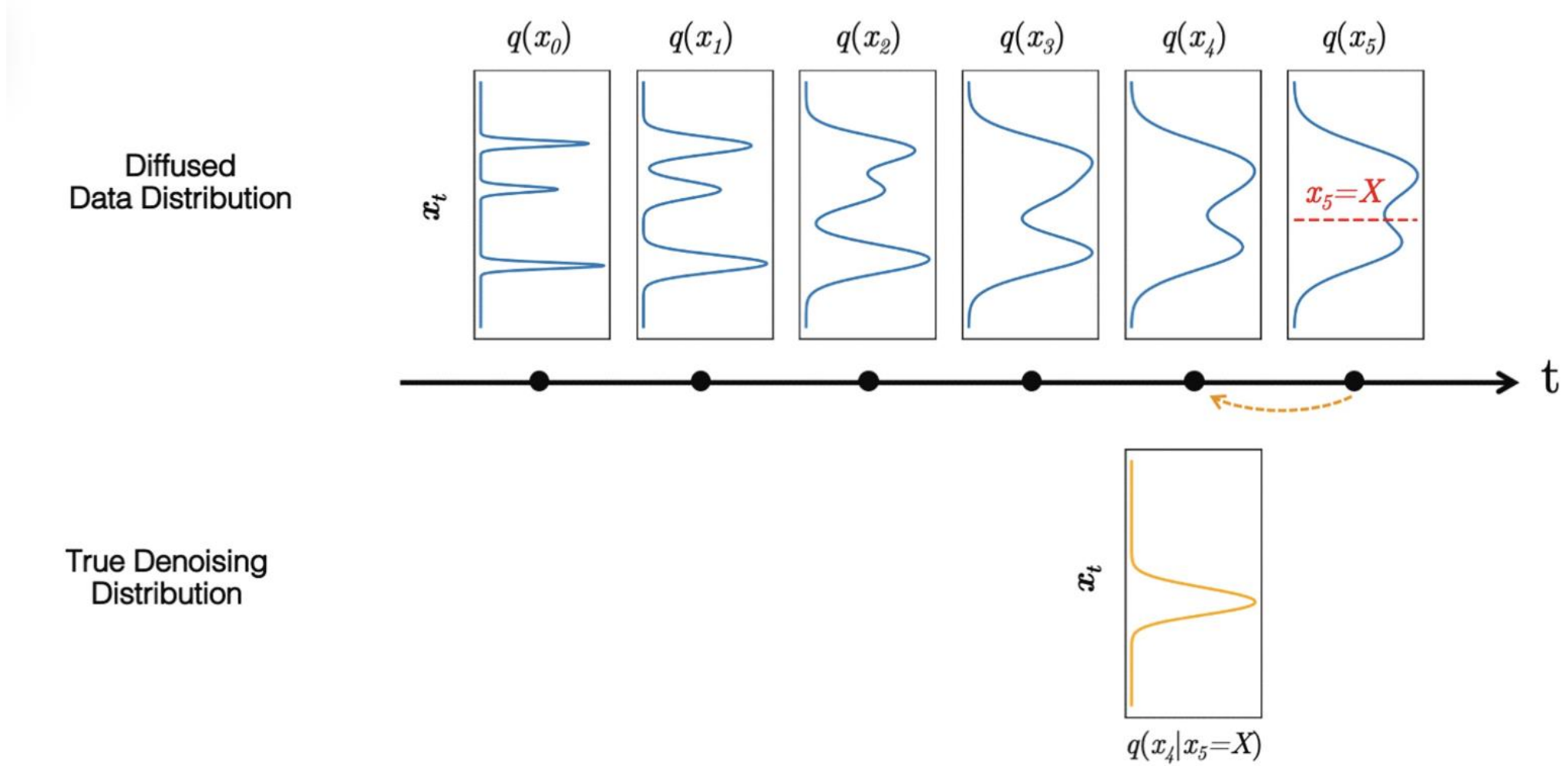
Learning Process

- The goal of a diffusion model is to learn the reverse denoising process to iteratively undo the forward process
- In this way, the reverse process appears as if it's generating new data from random noise



Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

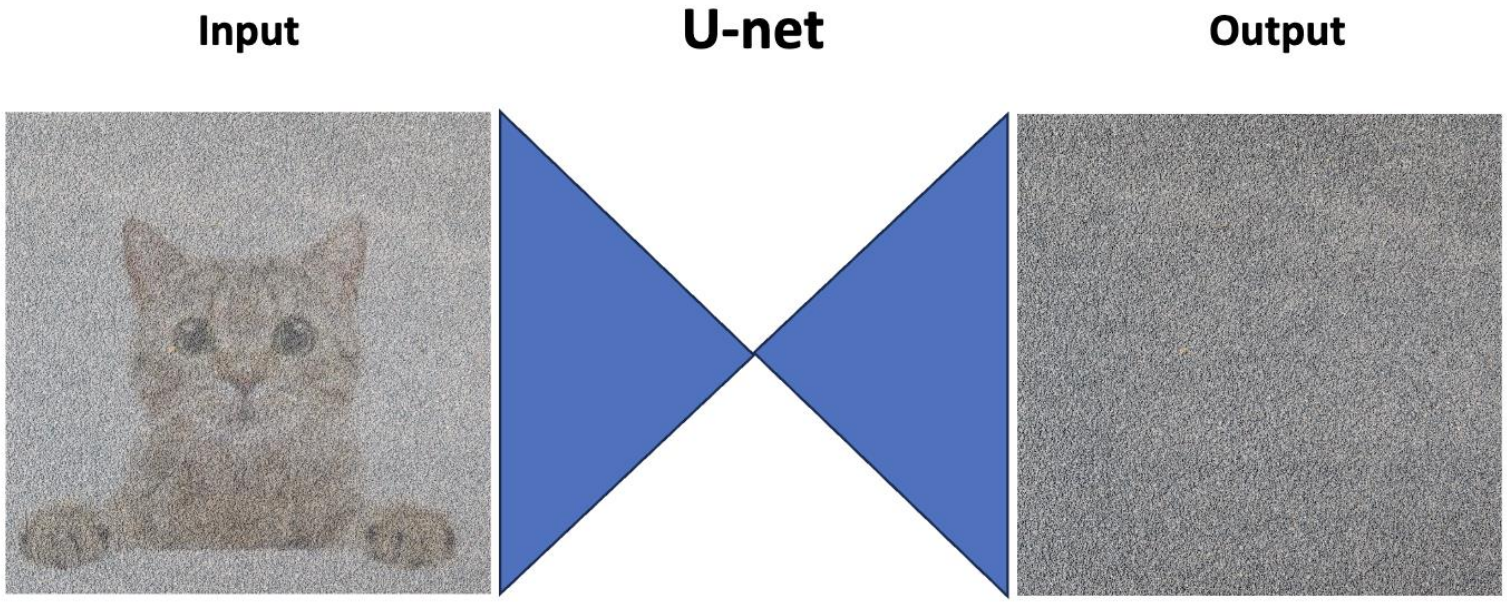
Distribution Changes Through Time



Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

Training

- Diffusion models predict the noises from the input images, usually by using U-Net



Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

Training

- Diffusion models predict the noises from the input images, usually by using U-Net

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

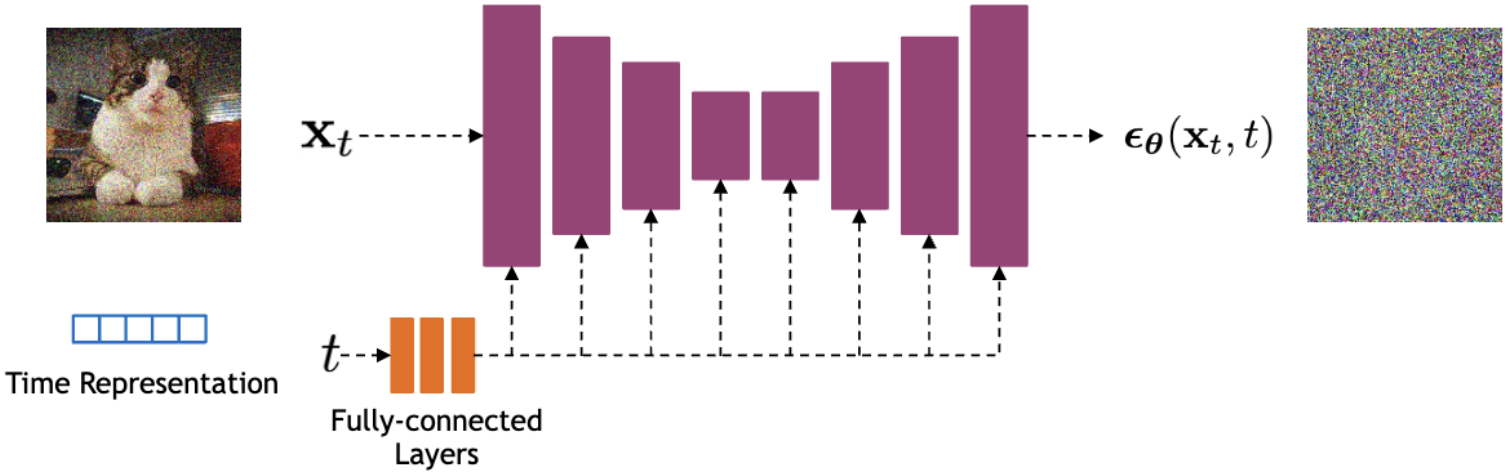
6: **until** converged

Random Noise

Model Predicted Noise

Model Details

- Diffusion models often use the U-Net architecture with ResNet blocks and self-attention layers to predict the noise $\epsilon_\theta(x_t, t)$



Time representation: sinusoidal positional embeddings or random Fourier features.

Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers. (see Dharivwal and Nichol NeurIPS 2021)

Image from https://cvpr2022-tutorial-diffusion-models.github.io/

U-Net Based Diffusion Models

Imagen
Saharia et al.

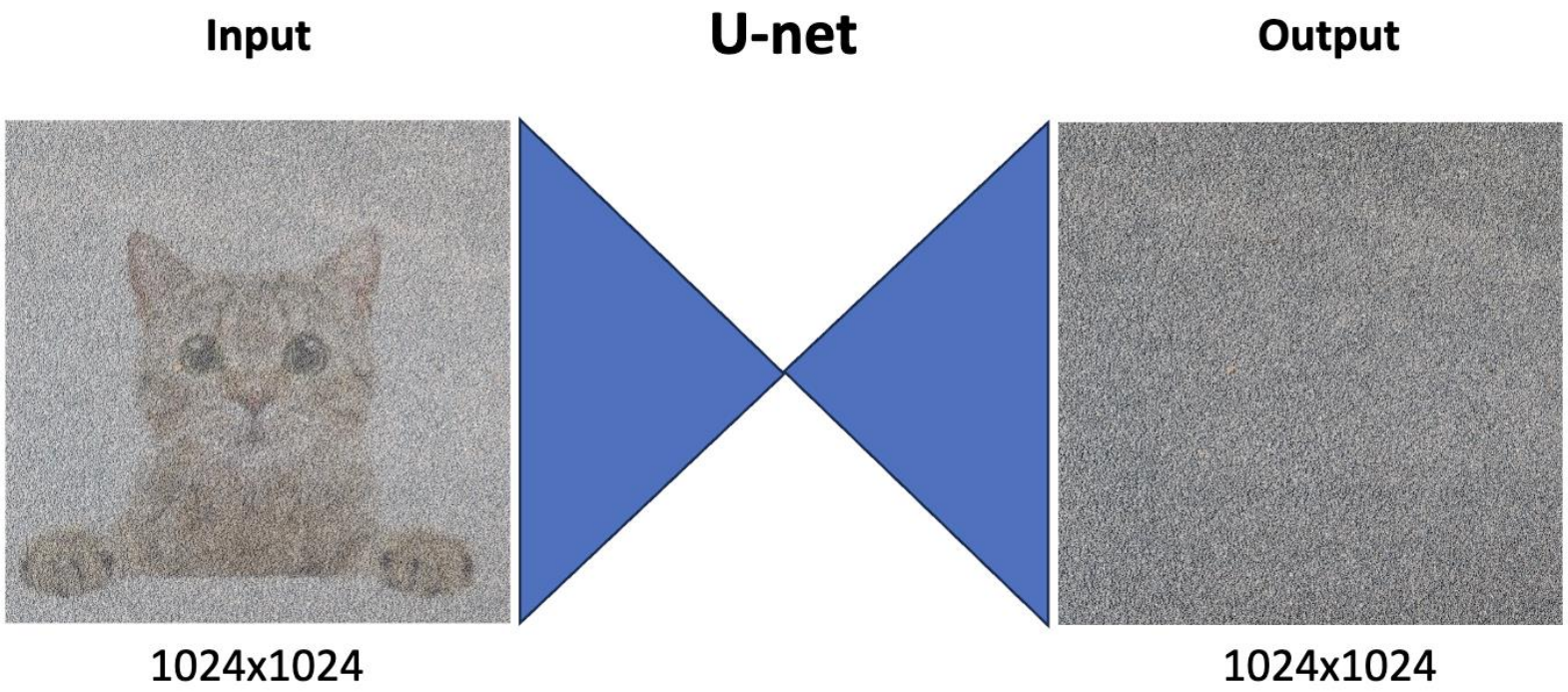
Stable Diffusion
Rombach et al.

eDiff-I
Balaji et al.

Image from https://cvpr2022-tutorial-diffusion-models.github.io/

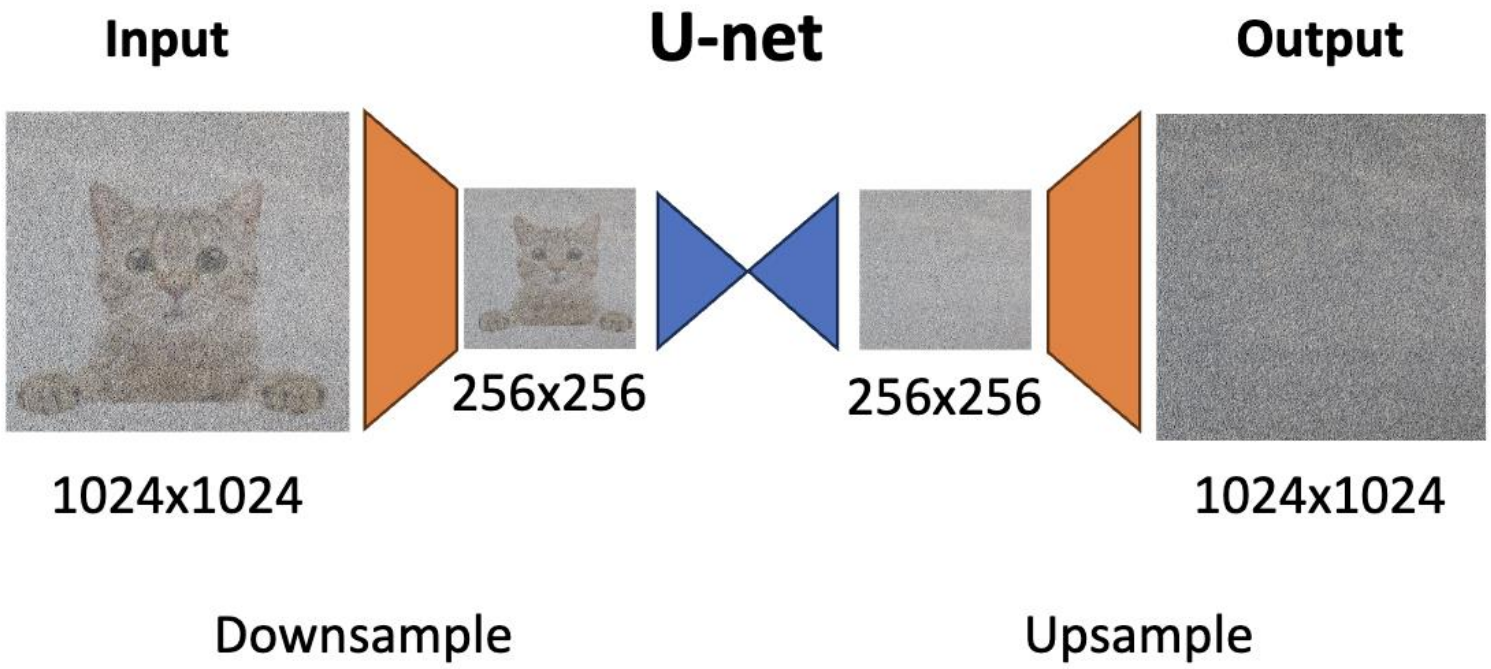
U-Net Problem

- Problem: operating in the input space is very computationally expensive



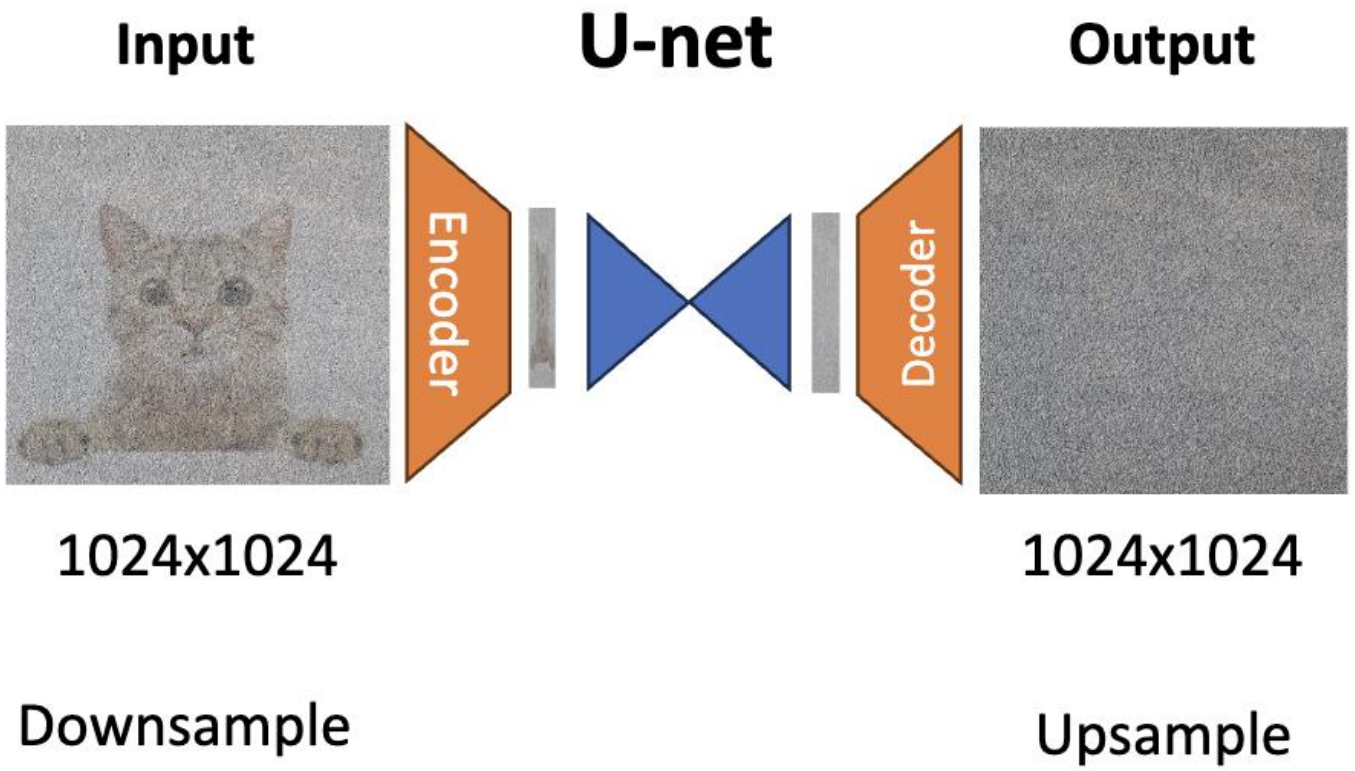
Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

Option #1: Generate Low-Resolution + Upsample



Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

Option #2: Generate in Latent Space



Slide from https://www.eecs.umich.edu/courses/eecs442-ahowens/fa23/slides/lec11-diffusion.pdf

Impressive Results

DALL·E 2

"a propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese"

IMAGEN

"A photo of a raccoon wearing an astronaut helmet, looking out of the window at night."

Image from https://cvpr2022-tutorial-diffusion-models.github.io/

Conclusion

- Variational Autoencoders (VAEs)
  - An encoder and decoder
  - Trained with variational inference
- Generative Adversarial Networks (GANs)
  - A discriminator and generator
  - Training a generator to fool the discriminator
- Denoising diffusion Decoders
  - Training a denoising network to recursively denoise a noising image starting from a noise